

Aether-Urban: Technical Architecture Report

Comprehensive Analysis of Colombo's Geospatial AI Workspace

1. Introduction

Aether-Urban is an advanced geospatial AI workspace designed for urban planning in Colombo, Sri Lanka. It integrates large language models (LLMs) with geographic information systems (GIS) to allow urban planners to query, analyze, and visualize spatial data using natural language. The system aims to drastically reduce the time needed to perform complex GIS analyses like proximity checks, zoning compliance, and area summaries.

2. System Architecture

The architecture of Aether-Urban consists of three primary layers:

1. **Frontend (Client-side):** Built using vanilla JavaScript and Leaflet.js, the frontend provides an interactive map interface. It connects to the backend via Server-Sent Events (SSE) to receive streaming GeoJSON data and map actions (pan, zoom, highlight) in real-time as the AI agent processes the request.
2. **API Layer (Backend):** A FastAPI Python application ('app.py') that exposes the chatbot and map data endpoints. It acts as a middleware, receiving user queries, parsing them, and interacting with the LLM. It includes a custom stream parser that extracts hidden 'tool-call' blocks from the LLM's response before sending the cleaned text to the user.
3. **AI & Geospatial Backend:** Uses Google's 'gemma-4-31b-it' (via Gemini API) or 'qwen2.5-coder' (via local Ollama) for reasoning. When the LLM decides a spatial analysis is needed, the backend triggers functions in 'spatial_tools.py'. This module utilizes GeoPandas, Shapely, and OSMnx to execute intersections, distance calculations, and routing.

Aether-Urban: Technical Architecture Report

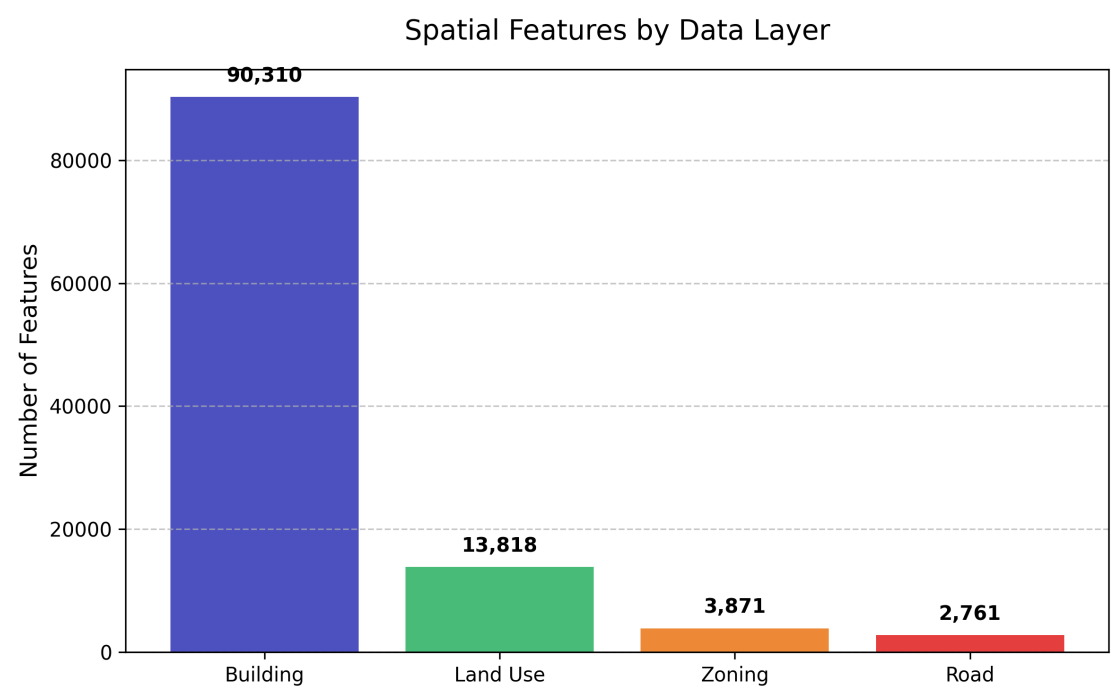
Comprehensive Analysis of Colombo's Geospatial AI Workspace

3. Data Pipeline & Performance Optimizations

Given the massive amount of geospatial data (over 100,000 polygons in total), several optimizations have been implemented to achieve millisecond response times:

- Parquet Caching: Shapefiles are automatically converted and cached as GeoParquet files, resulting in up to 5x faster read times.
- Pre-Projection: Spatial layers are pre-projected from WGS84 (EPSG:4326) to UTM (EPSG:32644) during startup and cached in memory. This eliminates the costly overhead of projecting large geometries during individual tool calls.
- Spatial Indexing (sindex): For bounding box filters and proximity analyses, spatial indices are used to instantly filter features rather than checking geometry intersections for the entire dataset.
- Dynamic Geometry Simplification: The backend applies the Douglas-Peucker algorithm to simplify complex polygons before transmission, dynamically calculating a tolerance based on the viewport span ($\text{span} / 1500.0$). This prevents browser thread locking when rendering large datasets.
- Viewport Restraints: Detailed layers like buildings (90k+ features) are restricted from loading fully until the user zooms in past a threshold (zoom level 15).

4. Spatial Data Feature Counts



Aether-Urban: Technical Architecture Report

Comprehensive Analysis of Colombo's Geospatial AI Workspace

5. Core Capabilities

Aether-Urban's spatial AI agent is equipped with the following tool-calling capabilities:

- Data Summarization: Get overall statistics, available columns, and unique attribute values for layers.
- Attribute Filtering: Find and map specific features (e.g., 'Commercial buildings', 'A-class roads').
- Spatial Intersections: Overlay layers to find intersections (e.g., finding buildings within a specific land use zone).
- Proximity Analysis: Buffer a target feature by a specific distance (e.g., '500m buffer around schools').
- Routing & Isochrones: Calculate walking paths and generate walking time polygons using OSMnx network graphs.
- Zoning Compliance: Verify if a proposed development coordinate complies with the underlying municipal zoning laws.